DESIGN AND IMPLEMENTATION OF A SELF BALANCING TWO-WHEELED ROBOT

21MHP302L - PROJECT REPORT

Submitted by

ALEN TITUS(RA2311038010012)
NIVED S (RA2311038010018)
ALBERT SYMAN C(RA2311038010030)
(Batch No.: 2)

Under the guidance of
Dr. Gangadevi R, M.E., Ph.D
(Associate Professor, Department of Mechatronics Engineering)

In Partial fulfilment for the degree

of

BACHELOR OF TECHNOLOGY

in

MECHATRONICS ENGINEERING (Robotics)

of

COLLEGE OF ENGINEERING & TECHNOLOGY



S.R.M. Nagar, Kattankulathur, Chengalpet District

May 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
(Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled "***Design and Implementation of a Self-balancing Two-wheeled Robot***" is the bonafide work of **Alen Titus(RA2311038010012) , Nived S (RA2311038010018) , Albert Syman C (RA2311038010030)**,who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported here in does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. Gangadevi R, M.E., Ph.D.
GUIDE
Associate Professor
Dept. of Mechatronics Engineering

SIGNATURE

Dr. T. Muthuramalingam, M.E., Ph.D. PROFESSOR &HEAD OF THE DEPARTMENT
Dept. of Mechatronics Engineering

Signature of the Internal Examiner 1

Signature of the Internal Examiner 2

ABSTRACT

The Self-Balancing Robot is a control systems-based project that focuses on designing and implementing a two-wheeled robot capable of maintaining its upright position autonomously. The system is based on the principle of an inverted pendulum, where continuous adjustments are required to keep the structure balanced.

In this project, orientation data is obtained using an inertial measurement unit (IMU) consisting of a gyroscope and an accelerometer. The sensor data is processed by a microcontroller, such as Arduino, which executes a feedback control algorithm. A Proportional–Integral–Derivative (PID) controller is implemented to calculate the error between the desired vertical position and the actual tilt angle of the robot. Based on this error, appropriate control signals are generated to drive the motors, ensuring real-time correction and stability.

The project demonstrates the integration of hardware and software components, including sensors, motor drivers, and embedded programming. It also highlights fundamental concepts such as sensor fusion, feedback control, and real-time system response. The successful implementation of the self-balancing robot provides practical insight into modern robotics and control engineering.

Furthermore, the principles applied in this system have significant real-world applications in areas such as autonomous robots and personal transportation devices like the Segway. This project thus serves as a foundational model for understanding and developing advanced intelligent systems.

ACKNOWLEDGEMENTS

It has been a great honor and privilege to undergo **B.Tech** in **MECHATRONICS** at **SRM Institute of Science and Technology**. We are very much thankful to the **Department of Mechatronics**, **SRM Institute of Science and Technology** for providing all facilities and support to meet our project requirements.

We would like to thank our Head of Department **Dr. T. Muthuramalingam, M.E., Ph.D.** and our project guide **Dr. Gangadevi R, M.E., Ph.D.** who gave us a big helping hand during the entire project.

The success and final outcome of this project required a lot of guidance and assistance from many people and we are extremely fortunate that we have got this all along the completion of our project work. Whatever we have done is only due to such guidance and assistance and we would like to thank them for their kind support.

ALEN TITUS

NIVED S

ALBERT SYMAN C

TABLE OF CONTENTS

ABSTRACT ii

ACKNOWLEDGEMENTS iii

LIST OF FIGURES vii

LIST OF TABLES ix

ABBREVIATIONS x

LIST OF SYMBOLS xi

1.Introduction 1

1.1.Background of the project. 1

1.2.Challenges in existing system 2

1.3.Applications 3

1.4.Need of the project 4

2.Literature survey 5

2.1. Overview. 5

2.2. Control methods for Self-Balancing Robots. 6

2.3. Sensors and Angle Estimation. 6

2.4. Motor Control and Hardware. 7

2.5. Encoder-Based Feedback Systems. 8

2.6. Research Gaps and Motivation 8

3.Design and fabrication of robot 9

3.1. System Overview 9

3.2. System architecture 10

3.3. Component description 11

3.3.1 Arduino UNO (Controller Unit). 11

3.3.2 MPU6500 IMU Sensor. 12

3.3.3 DC Geared Motors with Encoders. 13

3.3.4 Motor Driver 14

3.3.5 Power Supply System 15

3.4. Mechanical Design 16

3.5. Circuit Design and Connections 16

3.6. Fabrication and Assembly 18

3.7. Design Issues and Challenges 19

4.Methodology 20

4.1. Overview 20

4.2. Working Principle. 20

4.3. Sensor Data Acquisition and Processing 21

4.4. Control Algorithm (PID Control) 21

4.5. Motor Control Strategy 22

4.6. Encoder Feedback and Closed-Loop Control 23

4.7. Algorithm Implementation 23

4.8. FLOWCHART 24

4.9. System Data Flow 25

4.10. Implementation Considerations 25

5.Result and Discussion 25

5.1. Overview of Results. 25

5.2. PID Tuning Trials and Observations 26

5.3. Graph results 27

5.4. Discussion. 31

6. Conclusion and Future scope 31

6.1Conclusion 31

6.2 Future Enhancement (with load cell feature) 32

Appendix A

A.1 Encoder tuning

A.2 PID tuning

References

LIST OF FIGURES

3.1 Self-Balancing Robot 10

3.2 System Architecture Block Diagram 11

3.3 Arduino Uno R3 12

3.4 MPU6500 13

3.5 Planetary Gear Motor 12v 14

3.6 Cytron Bidirectional controller for DC motor 15

3.7 Battery (2200 mah) 15

3.8 Circuit view 17

3.9 connection view 17

3.10 Side View of robot 18

3.11 top view 18

3.12 Wheel Bracket 19

4.1 Flowchart 24

5.1 Graph for trial 1 28

5.2 Graph for trial 2 28

5.3 Graph for trial 3 28

5.4 Graph for trial 4 29

5.5 Graph for trial 5 29

5.6 Graph for trial 6 29

5.7 Graph for trial 7 30

5.8 Graph for trial 8 30

5.9 Graph for trial 9 30

5.10 Graph for trial 10 31

5.11 Graph for trial 11 31

LIST OF TABLES

5.1PID Tuning Trials and Observations 26

ABBREVIATIONS

MPU Motion Processing Unit

IMU Inertial Measurement Unit

MCU Microcontroller Unit

PID Proportional Integral Derivative

ALPHA Complementary filter coefficient

I2C Inter-Integrated Circuit

SDA Serial Data Line

SCL Serial Clock Line

ISR Interrupt Service Routine

LIST OF SYMBOLS

θ	Pitch angle
θ_acc	Pitch angle from accelerometer
θ_gyro	Pitch angle from gyroscope
θ_setpoint	Desired pitch angle
e(t), e(k)	Error signal (setpoint – measured value)
Kp	Proportional gain
Ki	Integral gain
Kd	Derivative gain
G(s)	PID controller transfer function
dt	Sampling time interval
Δ	Change in value
ω	Angular velocity

CHAPTER 1

INTRODUCTION

1.1 Background of the project

Self-balancing robots are a significant type of robotic system that illustrates the use of feedback control to achieve stability in an inherently unstable system. The systems use the inverted pendulum model, a classic control theory problem in which the system must dynamically maintain its balance. Because of its inherent instability, the system needs to be sensed, processed and moved precisely and rapidly.

With ongoing advancements in embedded systems and sensor technologies, the development of self-balancing robots has become more practical and efficient. The Arduino UNO microcontroller offers an affordable platform to implement control algorithms. Likewise, inertial measurement units (IMUs) such as the MPU6500 combine accelerometers and gyroscopes to generate orientation and angular velocity, which are needed to control balance.

But sensors have limitations. Accelerometers are accurate over long periods of time but prone to vibration and noise, and gyroscopes are accurate for short periods of time but drift. These drawbacks are overcome by combining different sensor measurements (sensor fusion) using a complementary filter to get a reliable measure of the tilt angle.

Self-balancing is usually implemented with a Proportional–Integral–Derivative (PID) controller. The controller determines the error between the upright position and the tilt angle, and produces control signals to the motors to re-balance the robot. The motor drive system, which is connected to the motors via a dual motor driver (e.g. Cytron MDD3A), accordingly varies the speed and direction of the motors.

For practical systems, other hardware elements such as incremental encoders are employed. The encoder readings are used to measure the wheel rotation and can be used

to enhance control and eliminate drift. But systems are subject to many non-ideal effects, such as sensor noise, electrical noise, motor deadband, and hardware asymmetry. This demands a consideration of both hardware and software techniques.

This project extends these basic ideas to create a self-balancing robot. This project aims to provide not only an implementation of control algorithms but also ways to overcome engineering issues to ensure a stable and robust system.

1.2 Challenges in existing system

Despite extensive research on self-balancing robots, they encounter various challenges during implementation that impact their stability, accuracy and performance. These challenges stem from constraints in sensor and actuator technologies, and the surrounding environment, making balancing a difficult task.

First, sensors can be inaccurate and noisy. The inertial measurement unit (IMU) that uses both accelerometer and gyroscope measurements can be disturbed. Accelerometers can be jolted by vibrations and other disturbances, and gyroscopes can drift. This leads to poor tilt angle calculation if not well-filtered.

The other problem is the unstable signals in encoder feedback systems. Encoder signals can be low voltage, floating, and prone to electrical noise, resulting in false pulse detection and inaccurate motor motion feedback. This affects the speed and position estimates.

Motor non-linearity is also a significant issue. DC motors generally have dead zones, where low PWM commands do not result in movement. And variations in motor properties lead to asymmetrical response, so it's hard to control consistently. This limits the control algorithm's effectiveness and may cause instability.

The system also experiences electrical noise and interference, particularly when multiple elements (motors, drivers, sensors) are used. Poor wiring and grounding practices can exacerbate these problems, affecting the quality and robustness of the system.

Real-time processing limitations of embedded systems such as the Arduino UNO are also a challenge. The processing power and timing accuracy can impact the control system's responsiveness, particularly when balancing sensor fusion, PID calculations and interrupt-based encoder reading.

The last challenge is tuning the PID controller. The choice of proportional, integral and derivative gains is iterative and relies heavily on the system's dynamics. Tuning is critical to avoid oscillations, sluggishness or instability.

Overcoming these challenges is crucial to designing a stable and safe self-balancing robot and is a major emphasis of this project.

1.3 Application

Self-balancing robots are used in numerous applications because they can balance using feedback and control systems. The technology developed in this project can be applied to various technologies and engineering systems.

A key application is in personal mobility solutions, like self-balancing scooters and electric transportation devices. Such devices also employ concepts of sensor integration and control for rider safety.

Self-balancing systems are also employed in mobile robotics for designing agile and lightweight robots that can operate in dynamic environments. Their two-wheel balance enables them to navigate more easily than conventional robots with multiple wheels.

Self-balancing systems are also used in industrial automation and material handling systems, where balance and stability are essential. They are used in robots for material handling and those that operate in tight spaces.

Finally, self-balancing robots are used in educational and research settings to learn about the design of control systems, robots, and embedded systems. They are often used in lab settings to illustrate control algorithms like PID, sensor fusion and real-time data processing.

Also, they are applicable to assistive robots and service robots, where stability plays a crucial role in interacting with humans. The concepts can also be applied to self-driving robots and package delivery robots.

Moreover, the principles applied to self-balancing robots are relevant in cutting-edge areas, such as aerospace and defense technologies, in aspects like stabilization and attitude control.

In summary, the project offers an introduction to technologies commonly found in many engineering applications, and is therefore relevant to both education and industry. 4

1.4 Need of the project

Building a self-balancing robot is crucial for the application of basic principles in control systems, robotics and embedded systems. Many contemporary engineering systems use real-time control and feedback for accurate control, so it is necessary to have practical experience in these techniques.

The main need of the project is to address the transition from theory to practice. Theoretical knowledge of PID control, sensor fusion, and feedback mechanisms is easily understood but in practice these will encounter issues such as noise, instability and hardware restrictions. This project offers an opportunity to tackle these problems and find solutions.

The project is also essential because of the increasing need for smart and stable robotics systems in industry. Self-balancing cars, mobile robots and autonomous systems are based on similar concepts. Knowing how to design and control these systems is essential for future robotics and automation technologies.

There is also a need to acquire skills in system integration, where several components like sensors, actuators and microcontrollers need to be integrated seamlessly. This involves real-time data handling, signal conditioning and control strategies.

Moreover, the addition of encoder feedback underlines the importance of precise motion control and feedback systems, critical in robotic systems. It also provides a

foundation for future developments including closed-loop speed control and system robustness.

In summary, this project is needed to gain practical experience, develop problem-solving abilities, and understand the principles of modern control and robotic systems, which are fundamental for both educational and industrial purposes.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview

Self-balancing robots have become a popular topic in robotics research as they provide an example of a control problem known as the inverted pendulum. The inverted pendulum is an inherently unstable system, which means it cannot balance itself without constant observation and adjustments. As a result, self-balancing robots are commonly used as a test bed for control systems, embedded systems programming, and sensor technologies.

Throughout the decades, numerous methods have been explored to balance the robot using control strategies, sensors and hardware design. In the past, such robots were constructed using large, expensive components, but with the advent of microcontrollers like the Arduino and cheap IMU sensors, it's easier to build smaller and cheaper self-balancing robots.

However, achieving stable balancing in practical situations still remains elusive. Sensor noise, drift, motor characteristics and environmental disturbances are some of the factors causing non-ideal behavior. Thus, it is important to review the techniques used in existing works and their shortcomings in order to design a system. In this chapter, an overview of the current control and sensor techniques, and challenges of self-balancing robots are discussed.

2.2 Control methods for Self-Balancing Robots

Stability of a self-balancing robot largely depends on the control techniques used. The primary goal of the controller is to reduce the error between the desired upright posture and the actual posture of the robot. The Proportional-Integral-Derivative (PID) control is the most common control strategy used because it is easy to design, implement and understand.

The PID controller comprises three terms. The proportional term provides an output proportional to the instantaneous error, enabling the system to respond rapidly to changes. The integral term integrates error over time and helps eliminate steady-state error, allowing the robot to stand upright in the long term. The derivative term anticipates errors based on their rate of change, thereby reducing oscillations and enhancing system stability.

While the PID control is straightforward, the tuning of the gain parameters (Kp, Ki, Kd) can be complex and of great importance. Incorrect tuning may result in instability, oscillations or sluggish behaviour. Different tuning strategies are adopted such as trial-and-error and Ziegler-Nichols methods.

Besides PID control, various other control strategies have been investigated, including Linear Quadratic Regulator (LQR), fuzzy logic control, and adaptive control. These controllers provide enhanced performance, stability and disturbance rejection. But they need mathematical modeling of the system and involve complex calculations, which are often not feasible in simple and cheap applications.

This makes PID control the most popular choice for practical self-balancing robots, particularly in academic and prototype projects.

2.3 Sensors and Angle Estimation

To keep the robot balanced, it is vital that the tilt angle can be accurately estimated. Self-balancing robots usually use an Inertial Measurement Unit (IMU), which consists

of an accelerometer and a gyroscope. They offer complementary data on the robot's tilt and acceleration.

The accelerometer detects acceleration in various directions and can be used to calculate the tilt angle with respect to gravity. It offers accurate long-term readings but is very sensitive to vibration and abrupt changes of motion, introducing noise.

The gyroscope measures the rate of change of angle, and offers accurate and smooth information over short times. But when it is integrated to get angle, the small errors add up causing drift.

To address these issues, sensor fusion methods are employed to leverage the benefits of each sensor. A popular technique is the complementary filter, which is simple to implement and effective. It combines the accelerometer and gyroscope readings using a high-pass filter to the gyroscope and a low-pass filter to the accelerometer to produce a robust and accurate angle measurement.

Other more sophisticated methods include the Kalman filter. The Kalman filter offers optimal filtering by taking into account the noise and dynamics of the system, but it's more computationally intensive and harder to implement.

In summary, sensor fusion is essential for enhancing the accuracy of tilt estimation, and for stable and smooth control of the robot.

2.4 Motor Control and Hardware

The motors are used to produce corrective motion to maintain the robot's balance, and play a critical role in system stability. Real-world DC motors have non-linear characteristics, making them hard to control. Perhaps the most common problem is the dead zone, where the motor doesn't respond to small values of PWM. This makes it hard to control the motor around the balance point.

Another problem is the disparity between the left and right motors. Slight variations in the motors' characteristics can cause imbalanced movements and instability. This needs to be calibrated and tuned to achieve stable control.

Motor direction problems can also stem from wiring or logic errors, resulting in the robot moving in the opposite direction to correct its tilt. These errors are fixed using software.

Apart from motor faults, hardware faults such as electrical noise, lack of proper grounding and crosstalk can affect sensor and encoder performance. Poor wiring and filtering can cause disturbances.

Techniques to overcome these issues include setting minimum PWM thresholds, motor response calibration, and grounding and noise filtering. Such considerations are crucial for ensuring robust and reliable performance in practice.

2.5 Encoder-Based Feedback Systems

Encoders are used to track the wheel rotation and give feedback on the robot's motion. For self-balancing robots, quadrature encoders are often used since they provide information on both the speed and direction of rotation. This provides more information for control.The encoders can be used to calculate the speed of the motor, which is useful in tuning the control loop and increasing stability. It also allows for future features like position tracking, trajectory planning and autonomous driving.

But the use of encoders comes with challenges. Encoder signals are typically low amplitude and prone to noise and interference, particularly in electrically challenging environments. These can result in errors or lost pulses.To increase accuracy, pull-up resistors are incorporated to strengthen the signal and interrupts are used to count the pulses. Debouncing and filtering techniques are also used to eliminate noise.Effective use of encoders improves the overall system and offers better control, particularly in complex applications.

2.6 Research Gaps and Motivation

While considerable research has been conducted on self-balancing robots, there remain some issues. A large part of the current research concentrates on the theoretical aspects

and simulations, while ignoring practical issues in real-world experiments. Problems related to sensor noise, drift, motor non-linearity and hardware constraints are not always considered.Furthermore, the development of stable real-time control with low-cost hardware remains a challenge. There is a compromise between system cost, complexity and performance. Complex control techniques can enhance system stability but demand more computing power and design effort.

A further limitation of current systems is the absence of hardware-level optimisations such as signal conditioning, grounding and motor calibration that impact system performance.The project is driven by the need to bring theory into practice. The goal is to build a self-balancing robot that not only utilises good control methods but also overcomes practical problems through hardware and software integration.Through the use of sensor fusion, PID control and troubleshooting, this project offers insight into real-time control systems and helps to build more stable and reliable robots.

CHAPTER 3

DESIGN AND IMPLEMENTATION

3.1 System Overview

The self-balancing robot is a two-wheel inverted pendulum mechanism, inherently unstable, and must be kept in balance by means of constant feedback control. The system is a combination of sensing, processing and actuation units to provide real-time balancing. Fig. 3.1 shows the fabricated self-balancing robot used for the implementation of the system. The MPU6500 IMU sensor is used to measure the tilt of the robot at all times. The Arduino UNO processes this data and performs a control algorithm to decide on corrective action. Depending on this output, PWM signals are produced and fed to the motor driver which drives the DC motors. The motors vary in speed and direction to ensure that the robot is in an upright position.

The encoders also give feedback about wheel rotation to allow more precise movement control, and in order to enhance the stability of the system. All the system is intended

to be in real time with a low latency between sensing and actuation. Robustness in hardware and software integration makes it perform reliably and solve practical issues like sensor noise, motor non-linearity, and electrical disturbances.

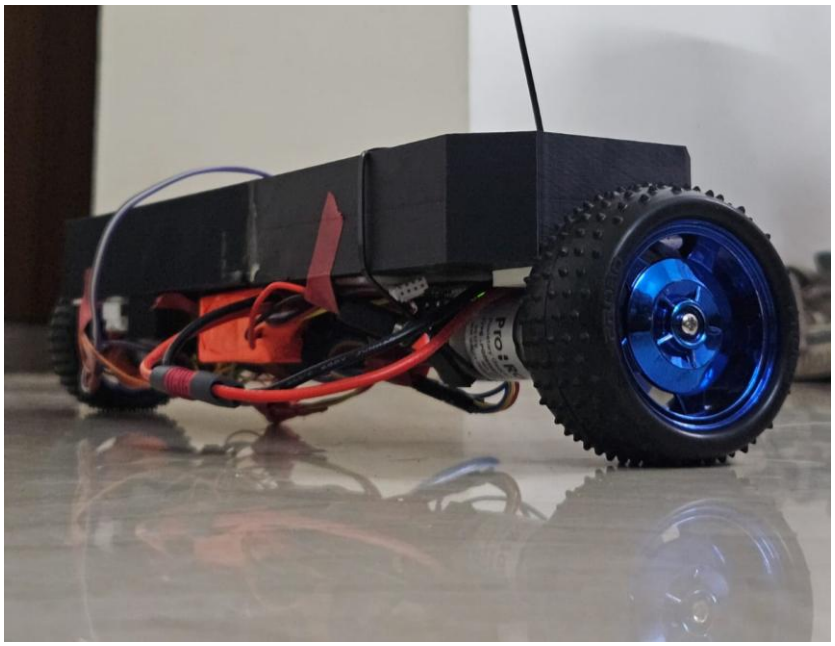


Figure 3.1: Self-Balancing Robot

3.2 System Architecture

System architecture is based on closed-loop control system. It is composed of three primary blocks: sensing, control and actuation. Fig. 3.2 illustrates the system architecture of the self-balancing robot. The sensing unit incorporates MPU6500 IMU that enables real-time information on the orientation of the robot. This information is sent to the Arduino via I2C communications. This data are then processed by the control unit (Arduino) and corrected accordingly with a control algorithm. The motor driver and DC motors that perform the corrective motion are found in the actuation unit.

Monitoring encoder feedback is carried out continuously and utilized to optimize control actions. Closed-loop design of the system makes sure that it corrects any changes in position other than the upright posture.

The component description and selection are described in 3.3.

.

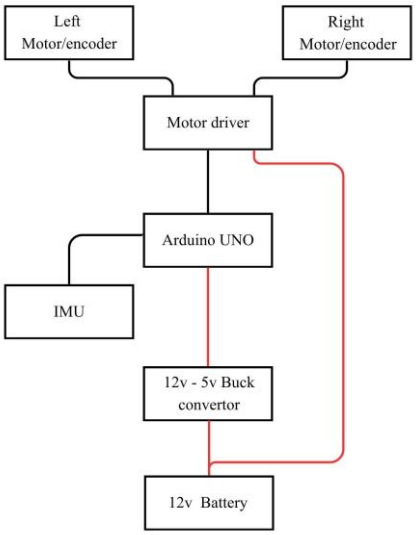


Figure 3.2 System Architecture Block Diagram

3.3 Component description

3.3.1 Arduino UNO (Controller Unit)

The Arduino UNO will act as the heart of the system. It does the sensor data acquisition, control algorithms and motor control output signals generation. Fig. 3.3 shows the Arduino UNO used as the controller unit in the system. The microcontroller is pegged on ATmega328P and it runs at a 16 MHz clock. It offers adequate computational power to support real time control like balancing. Arduino is chosen because it is easy to use, contains libraries, and can be used with many sensors and actuators. It is also capable of supporting interrupt-based processing which is needed to provide correct encoder readings. The next benefit of Arduino is that it has a predictable timing behavior that is essential in using control loops with a steady sampling rate.



Figure 3.3: Arduino Uno R3

3.3.2 MPU6500 IMU Sensor

The IMU sensor is MPU6500, which measures the tilt and motion of the robot. It is composed of a 3-axis gyroscope and 3-axis accelerator. Fig. 3.4 shows the MPU6500 IMU sensor used for measuring tilt and motion of the robot. Accelerator is used to measure linear acceleration and give information regarding the orientation of robot relative to gravity. But it is delicate to vibrations and external disturbances. Angular velocity is measured by the gyroscope which gives accurate short-term measurements and poor long-term drift. These two sensors are complementary to each other and the combination of them offers complementary information needed to estimate the tilt accurately. The MPU6500 was chosen based on its high precision, short response time, and small size. It interacts with the Arduino via the I2C protocol, making it easy to wire and use pins.



Figure 3.4: MPU6500 Acceleration and gyroscope

3.3.3 DC Geared Motors with Encoders.

The system employs DC geared motors with built-in quadrature encoders to work out the precise control of motion. The reason why these motors are chosen is that they are capable of delivering high torque at low speed which is needed in keeping a self-balancing robot in balance. Fig. 3.5 shows the planetary gear DC motor with inbuilt encoder used in the system. The gear reduction mechanism provides a smoother motion and enables the robot to react to disturbances.

The encoders are integrated, producing pulses that are proportional to the motor shaft rotation. The speed and direction of the wheels are determined with the help of these pulses. It also has high precision in the feedback of the rotational speed and direction since the quadrature encoding is employed thus enhancing overall control performance.

The importance of encoder feedback with the closed-loop system is that it enables a controller to measure the movement of the actual motor and correct it accordingly. This enhances stability and better reaction to tilt change.

The inbuilt encoders also make the design less cumbersome since fewer other components and wiring are required. Caution should be observed though, when dealing with signal noise by grounding and interrupt processing. All in all, DC geared motors

with encoders would be a dependable and efficient approach to getting precise and steady motions control in the robot.



Figure 3.5 Planetary Gear Motor 12v

3.3.4 Motor Driver

The motor driver is a connection between the Arduino and the motors. Because the Arduino is not capable of providing the current needed to directly power the motors, the motor driver is employed to boost the control signals. Fig. 3.6 shows the Cytron bidirectional motor driver used for controlling the DC motors. The Arduino sends it PWM and direction signals which it translates into a corresponding voltage and current needed by the motors. It also protects against overcurrent and voltage fluctuations, providing safe operation of the system by the motor driver. A dual-channel motor driver is chosen to independently drive the two motors which enables fine control of the movement.

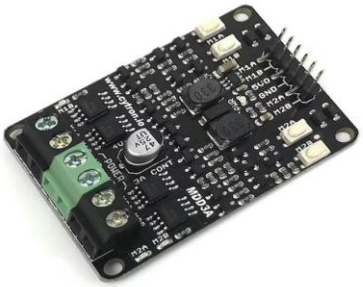


Figure 3.6 Cytron Bidirectional controller for DC motor

3.3.5 Power Supply System

A rechargeable battery is used to provide the necessary voltage to the motors and electronic components, which power the system. Fig. 3.7 shows the rechargeable battery used as the power source for the system. A buck converter or voltage regulator is required to reduce the voltage to safe levels of the Arduino and sensors. Voltage drops may cause poor system performance and this requires proper power management to maintain stability and to avoid voltage drops.

.



Figure 3.7 Battery (2200 mah)

3.4 Mechanical Design

The robot's mechanical design ensures stability and balance. The robot is a two-wheel robot with a vertical body.It is important to get the weight above the wheel axis. An imbalance in weight will cause instability.A lightweight and stiff chassis is required to reduce vibrations and chassis deflection.Components like the battery, controller and sensors are strategically placed for weight distribution.

3.5 Circuit Design and Connections

The circuit is designed for efficient communication and power supply. Fig. 3.8 shows the circuit diagram of the self-balancing robot system, and Fig. 3.9 shows the assembled hardware setup of the robot. The MPU6500 is connected to the Arduino via I2C (SDA and SCL pins). PWM pins are used to control motor speed and digital pins to control direction. Encoders are connected to interrupt pins for pulse counting.

All components are grounded to minimise noise. Decoupling capacitors may be used to stabilize voltage levels.

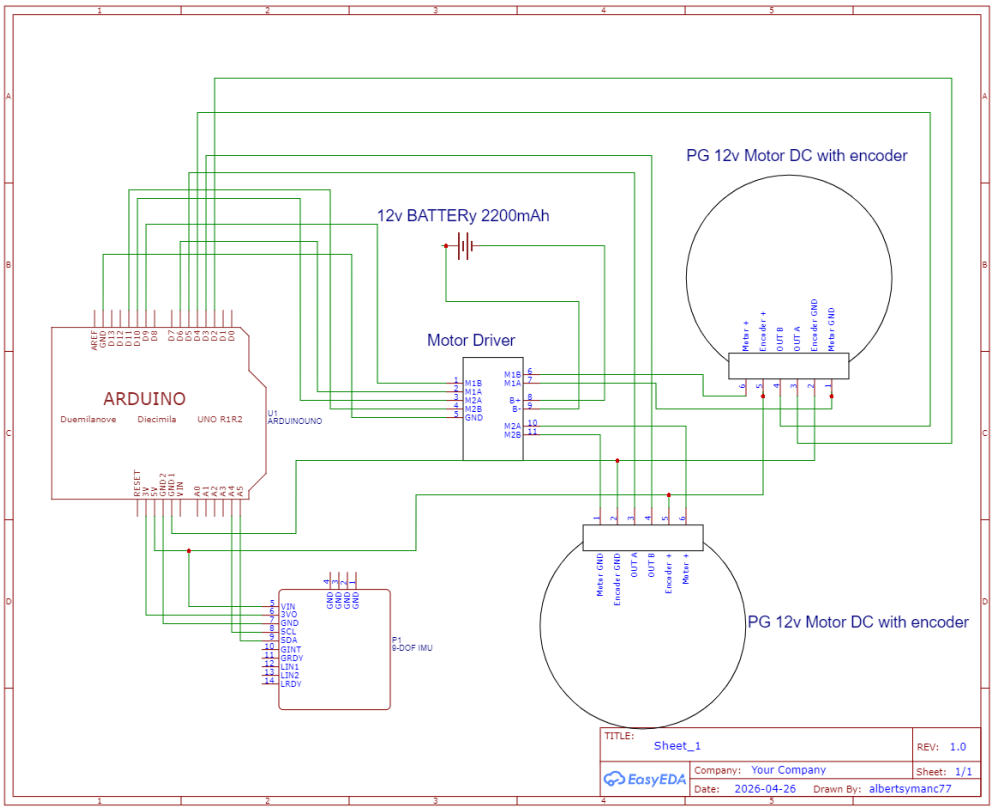


Figure 3.8 Circuit view

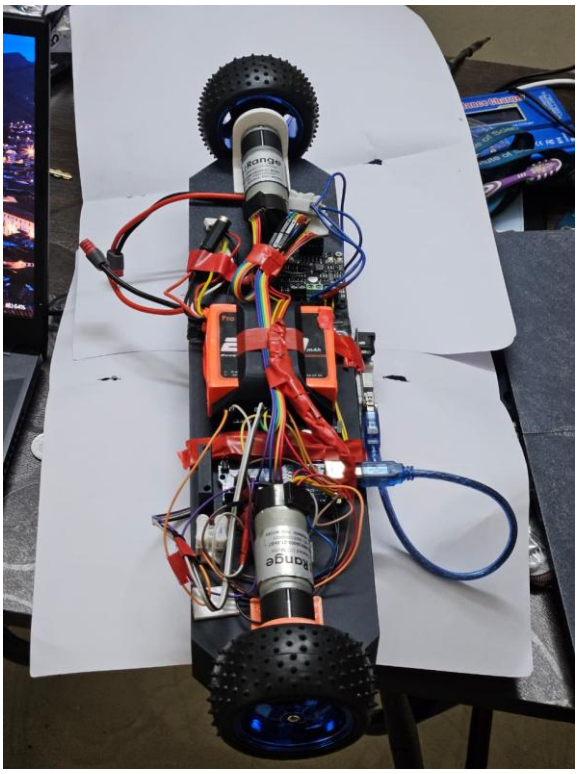


Figure 3.9 connection view

3.6 Fabrication and Assembly

The assembly process involves mounting all the mechanical and electrical components on the chassis. Fig. 3.10 shows the side view of the robot, Fig. 3.11 shows the top view, and Fig. 3.12 shows the wheel bracket used in the design. Motors are fixed to avoid misalignment and wheels are mounted correctly for proper movement. The IMU sensor sensor is securely mounted to reduce vibration. Electrical connections are neatly laid out to prevent interference. The robot is tested as it is assembled to ensure component functionality.

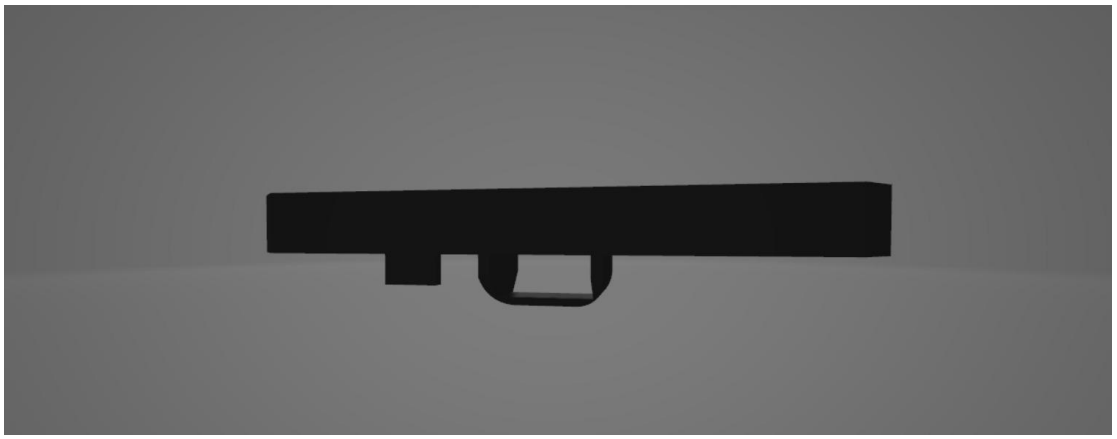


Figure 3.10 Side View of robot

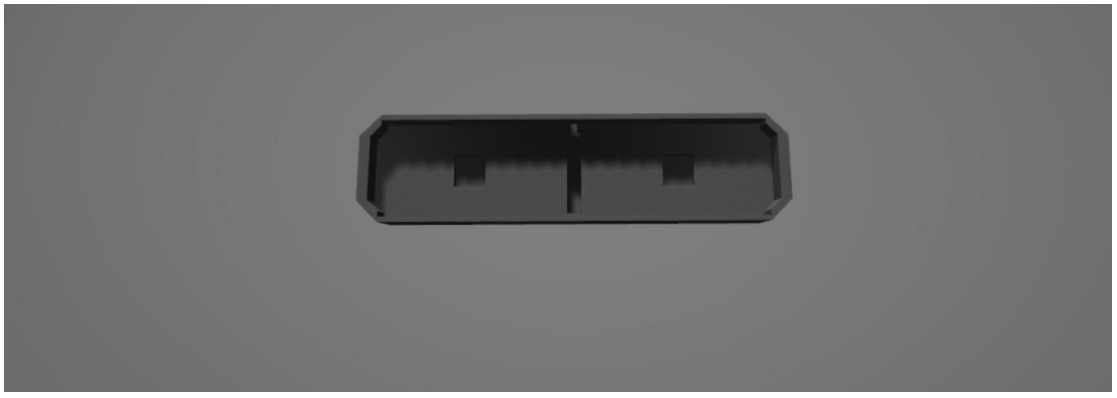


Figure 3.11 top view



Figure 3.12 Wheeel Bracket

3.7 Design Issues and Challenges

A number of challenges were faced during the design and implementation of the robot.First was the motor dead zone, where motors do not respond to low PWM values. This was overcome by setting a minimum level.Tilt estimation was impacted by sensor drift and noise, which was addressed through filtering.Weight distribution was an issue that was resolved by repositioning components.Encoders were susceptible to electrical noise and interference, solved by pull-up resistors and grounding.These issues demonstrate the need for practical aspects in system design.

CHAPTER 4

METHODOLOGY

4.1 Overview

This chapter describes the approach that was used in the process of implementing the self-balancing robot. The design of the system is based on a real-time closed-loop control system wherein the upright position of the robot is maintained by continuous feedback of the sensors.

The methodology consists of several steps such as sensor data collection, signal processing, implementation of the control algorithm, and actuation of the motor. The IMU sensor is the MPU6500 sensor which measures the orientation of the robot and the data is processed in the Arduino UNO to determine the corrective action that should be taken.It is a continuous loop system so that any change in position that is not vertical is corrected immediately. Combination of hardware elements and effective control logic enables the robot to perform stable and reliable balancing behaviour.

4.2 Working Principle

The self-balancing robot functions on the inverted pendulum principle in which the system is inherently unstable and the system needs to be corrected at all times in order to achieve equilibrium. This is unlike the stable systems where the slightest disruption can make the robot fall unless something is done to correct the situation.The sensor is the MPU6500 which constantly measures the tilt of the robot with respect to its vertical axis. The desired position of the robot is the 0 degree position which is the perfectly upright position. Anything to the contrary is considered to be an error.

As the robot is tilted forward, the center of gravity moves forward of the axis of the wheel. In order to offset this, the motors are pushed forward causing the base to move beneath the center of gravity and re-establishing a balance. In the same way, the motors act in the opposite direction to the rotation when the robot is tilted backwards, so that the position is corrected.This is done at a very rapid rate, usually in milliseconds, and

enables the robot to dynamically keep in balance even with disturbances like slight pushes or uneven surfaces.

4.3 Sensor Data Acquisition and Processing.

The MPU6500 IMU gives raw data of both accelerometer and gyro. The accelerator senses the gravity direction and gives values on the angle of tilt but it is influenced by the vibrations and external forces. Angular velocity is measured by the gyroscope and this is beneficial in the detection of fast changes in orientation but it is prone to drift with time.In order to have a good and steady estimate of the tilt angle, a complementary filter is applied. The filter is a combination of the stability of the accelerator in low frequency and the high-frequency precision of the gyroscope. The outcome is a smooth and trustworthy angle measurement that has the potential of being utilized as a control.The filtered angle is continuously updated and sensor data is read at a fixed time period. The sampling rate should also be maintained to achieve a stable performance of the system and proper control.

4.4 Control Algorithm (PID Control)

The balance of a robot is controlled by a Proportional-Integral-Derivative (PID) controller. The controller also compares the desired angle (setpoint) and the measured angle through the sensor by continually computing the error.The proportional term produces an output proportional to the existing error that allows the error to be responded quickly. The accumulating term builds up the historical errors and assists in removing a steady-state offset so that the robot does not lose its upright posture as it moves in time. The derivative term is a term that anticipates the error in the future on the basis of the rate of change in order to minimize the oscillations and enhance the stability of the system.The sum of these three elements is the control signal that defines the reaction that the motors are to provide. PID parameters must be tuned appropriately since tuning them improperly may cause instability, over oscillations or slow reaction.

Accelerometer Pitch

$\theta_acc = \tan^{-1}(ay / az) \times (180 / \pi)$

Gyroscope Conversion

$\omega = gx / 131$

Gyro Integration

$\theta = \theta + \omega \times dt$

Complementary Filter

$\theta = \alpha(\theta + \omega dt) + (1 - \alpha)\theta_acc$

Error Calculation

$e = \theta - \theta_setpoint$

Proportional Term

$P = Kp \times e$

Integral Term

$I \approx Ki \times \Sigma e$

Derivative Term

$D \approx Kd \times (e - e_prev)$

PID

$u(t) = Kp \times e(t) + Ki \times \int e(t)\, dt + Kd \times (de(t) / dt)$

Dead Zone

$-4° < \theta < -3°$

4.5 Motor Control Strategy

The PID controller output is mapped into Pulse Width Modulation (PWM) signals, for motor control. The PWM value controls the speed of the motors and the direction signal controls the direction of the robot (forward or backward).A large error results in a larger

PWM value, making the motors react rapidly to correct the tilt. As the robot nears the equilibrium position, the PWM value is reduced, preventing overshoots.A threshold minimum value is set to avoid the motor dead zone, allowing the motors to respond to low PWM values. This enhances the stability around the upright position.

4.6 Encoder Feedback and Closed-Loop Control

The motors' built-in encoders give feedback on wheel movement. The feedback is used to track the robot's movement and enhance control.The encoder pulses are counted using interrupts, which provide accurate measurement without losing pulses. This feedback helps identify any deviations between the actual and desired motor responses, allowing the controller to adjust its output to compensate for these errors.The robot system is a closed-loop system that involves feedback from the sensor, control action and the output to the motor. This closed-loop system allows the robot to adapt to different situations and disturbances.

4.7 Algorithm Implementation

The control algorithm runs in a loop in the microcontroller. The steps are as follows:

Start the system and sensor modules

Read raw data from the MPU6500 sensor

Use complementary filter to calculate tilt angle

Compute the angle error

Compute PID control output

Generate PWM signals

Send signals to motor driver

Read encoder feedback for motion monitoring

Repeat the loop continuously

This loop is executed at a high rate to achieve real-time operation.

4.8 Flowchart

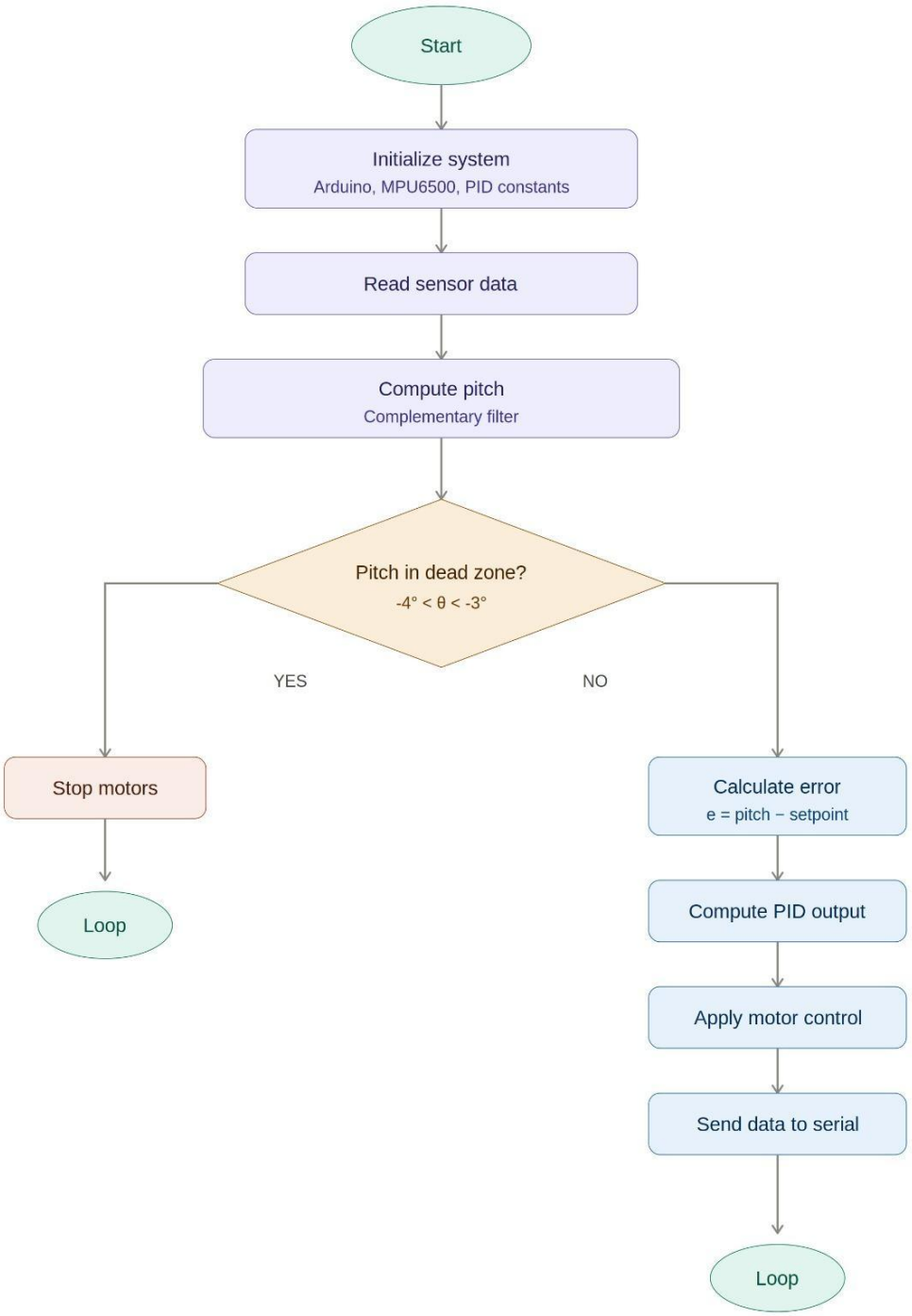


Fig 4.1 Flowchart

4.9 System Data Flow

There is a structured data flow in the system. Data from the IMU sensor is fed into the Arduino, which processes it and generates control signals. These are then sent to the motor driver to control the motors. The control is fine-tuned using encoder data.Such constant feedback allows the system to adapt to changes and remain stable.

4.10 Implementation Considerations

A number of issues were addressed during implementation. It was important to have a fixed sampling time for stable control. Filtering methods were used for sensor noise reduction.

Both motors were calibrated to have the same response and avoid motor imbalance. Dead zone compensation was applied to compensate for low speed response.Limits were also applied to avoid damage to the system due to high motor speeds. These factors were important for achieving stable performance.

CHAPTER 5

RESULT AND DISCUSSION

5.1 Overview of Results

This chapter shows what happened when we tuned the PID for the self-balancing robot. We looked at how well the system worked by changing the proportional (Kp), integral (Ki), and derivative (Kd) gains one at a time and in groups. The system's response was recorded and observed for each set of values using a graph.We looked at the system's response based on important performance metrics like stability, oscillations, overshoot, and settling time. We did some initial tests by using P, I, and D separately to see how they affected the behavior of the system. These tests helped us figure out what each control part could and couldn't do on its own.

We did more tests by mixing the PID parameters in different amounts. This process of trial and error helped find the best combination of values. The graphs that go with this

make it easy to see how the system reacts to different tuning conditions.The results in this chapter show how important it is to properly tune the PID in order to get stable and efficient control of the self-balancing robot.

5.2PID Tuning Trials and Observations

Trial No.	Kp	Ki	Kd	Observation	Figure No.
1	10	0	0	The system starts responding but shows continuous oscillations and does not settle properly. Stability is not achieved.	Fig 5.1
2	20	0	0	The response becomes faster, but oscillations increase significantly, making the system unstable.	Fig 5.2
3	45	0	0	The system becomes highly unstable with very large oscillations and fails to maintain balance.	Fig 5.3
4	0	6.7	0	The system responds very slowly and shows drifting behavior. It is not able to stabilize on its own.	Fig 5.4
5	0	15.6	0	Overshoot is observed and the system becomes unstable due to accumulation of error from the integral term.	Fig 5.5
6	0	31	0	The system becomes completely unstable and diverges rapidly due to excessive integral action.	Fig 5.6
7	0	0	5.2	Sligh seen, but the system does not correct it.	Fig 5.7
8	0	0	10	Damping improves slightly, but the system still fails to reach the desired position.	Fig 5.8

9	0	0	48.8	The system becomes very slow and unresponsive due to excessive damping.	Fig 5.9
10	7	8.8	0	The system shows improvement with reduced steady-state error, but oscillations are still present due to lack of derivative control.	Fig 5.10
11	5	8	3	The system achieves stable behavior with minimal oscillations, low overshoot, and quick settling time. This combination provides the best overall performance.	Fig 5.11

TABLE 5.1

5.3 Graph Results

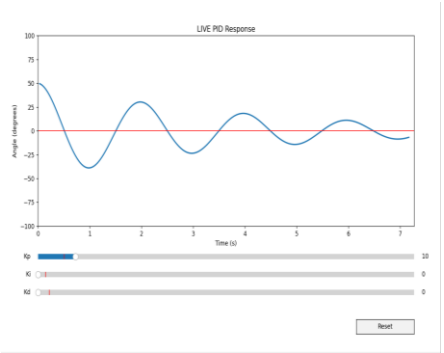


Fig 5.1

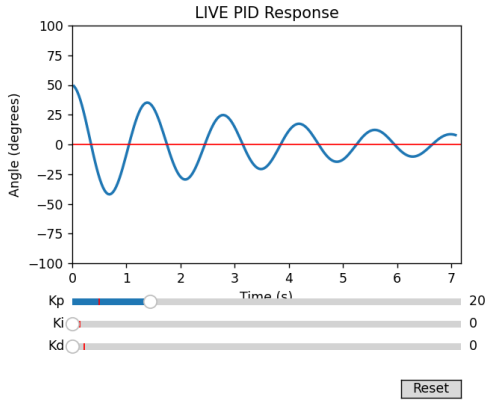


Fig 5.2

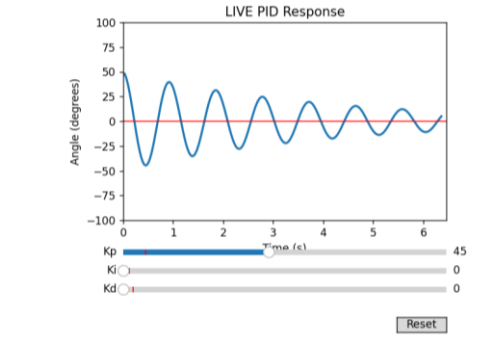


Fig 5.3

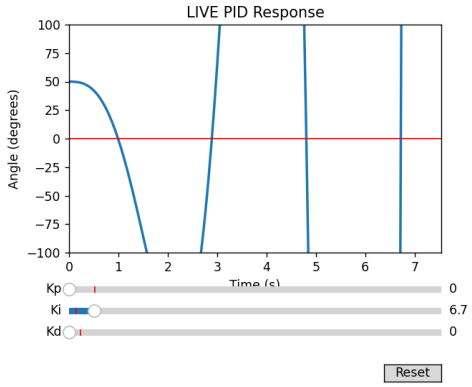


Fig 5.4

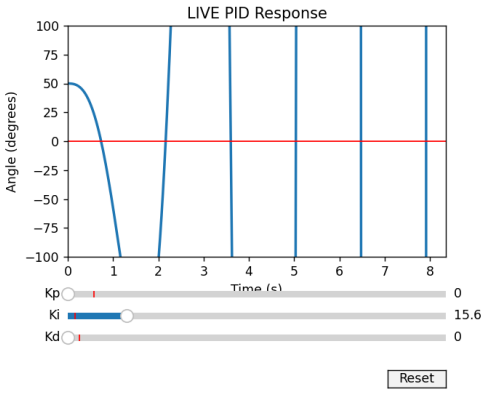


Fig 5.5

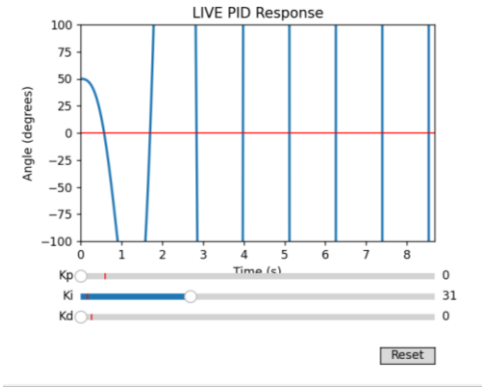


Fig 5.6

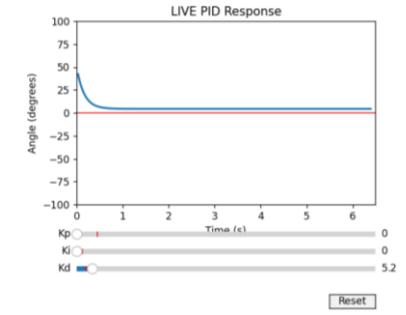


Fig 5.7

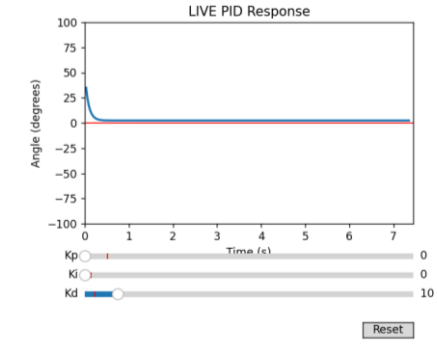


Fig 5.8

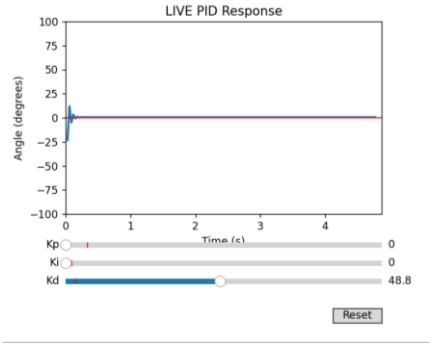


Fig 5.9

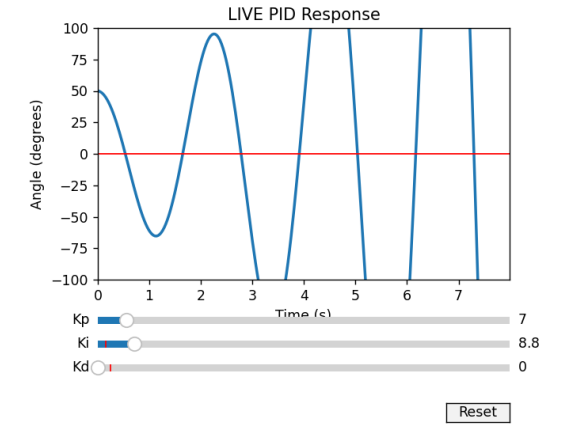


Fig 5.10

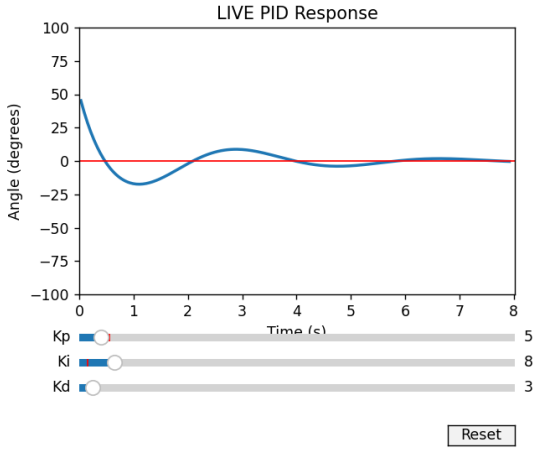


Fig 5.11

5.4 Discussion

The PID tuning process was done by changing the proportional, integral, and derivative gains and trying different things until it worked. At first, only proportional control was used, which caused a lot of oscillations and made things unstable.

Increasing the proportional gain made the response time faster, but it also caused too many oscillations. Adding the derivative term helped the system dampen better and cut down on oscillations. But the system still had small steady-state errors.Then, the integral term was added to get rid of steady-state error and make things more accurate. Some tests showed overshoot and instability because the integral gain was not set up correctly.We found the best PID values after trying a few times: Kp = 5, Ki = 8, and Kd = 3. The system behaved stably at these values, with little overshoot and a quick settling time.The results clearly show that a self-balancing robot needs all three PID parameters to be set correctly in order to have stable and efficient control.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project is a successful demonstration of a self-balancing robot that uses an Arduino UNO, MPU6500 Inertial Measurement Unit (IMU) sensor, DC geared motors with integrated encoders, and a dual motor driver. The design is inspired by the inverted pendulum and employs real-time closed-loop control.Sensor fusion (using a complementary filter) was used to determine the tilt angle, and the PID control algorithm was used to correct the balance. It could adapt to disturbances through motor adjustments and thus maintain balance.

The use of encoder feedback also improved the system's response by providing more accurate motion and control over the motor. Issues such as sensor noise, motor deadzone and signal drift were effectively resolved through calibration, filtering and

hardware modifications.In all, the project showcases the successful implementation of control system principles in a robotic system. It serves as a basis for the study of embedded systems, sensor technologies and real-time control, and a stepping-stone for future robotics projects.

6.2 Future Enhancement (with load cell feature)

While the existing system provides stable balancing, there are some potential enhancements and features.A potential future enhancement is to apply other control strategies like Linear Quadratic Regulator (LQR) or Kalman filtering to increase the stability and accuracy. These techniques may offer improved stability and accuracy over traditional PID control.The inclusion of wireless communication technologies, such as Bluetooth or Wi-Fi, using modules such as ESP32 can be used to remotely monitor and control the robot.

A crucial safety feature is the addition of a load cell sensor to detect a user or load. The load cell will be able to sense the weight on the robot. If the user or load is suddenly removed, such as in the case of a fall, the weight will decrease sharply. The controller can use this condition to shut off the motors to prevent the robot from continuing to move and potentially causing injury. This function enhances the safety and robustness of the system, particularly for applications involving interaction with a user.The system can be further improved by adding autonomous navigation functions, enabling the robot to walk and balance at the same time. This can include path planning and obstacle avoidance.

Other enhancements can include a refined mechanical design for better stability, use of more accurate sensors to eliminate noise and better power management for longer battery life.These improvements can enhance the capability and performance of the robot, and make it more useful in various high-tech and practical robotic applications.

APPENDIX A

CODE

A.1 Encoder tuning

```
/* -------- PINS -------- */
#define ENC_L_A 2    // INT0
#define ENC_L_B 4

#define ENC_R_A 3    // INT1
#define ENC_R_B 6

/* -------- COUNTS -------- */
volatile long leftCount  = 0;
volatile long rightCount = 0;

/* -------- SETUP -------- */
void setup() {
  Serial.begin(115200);

  // Use pull-ups (MANDATORY for Hall encoders)
  pinMode(ENC_L_A, INPUT_PULLUP);
  pinMode(ENC_L_B, INPUT_PULLUP);
  pinMode(ENC_R_A, INPUT_PULLUP);
  pinMode(ENC_R_B, INPUT_PULLUP);

  // Attach interrupts ONCE
  attachInterrupt(digitalPinToInterrupt(ENC_L_A), leftISR, FALLING);
  attachInterrupt(digitalPinToInterrupt(ENC_R_A), rightISR, FALLING);

  Serial.println("ENCODER TEST STARTED");
  Serial.println("Rotate wheels by hand");
}
```

```
/* -------- LOOP -------- */
void loop() {
  static unsigned long lastPrint = 0;

  if (millis() - lastPrint >= 300) {
    lastPrint = millis();

    long l, r;
    noInterrupts();
    l = leftCount;
    r = rightCount;
    interrupts();

    Serial.print("LEFT: ");
    Serial.print(l);
    Serial.print("   RIGHT: ");
    Serial.println(r);
  }
}

/* -------- ISRs -------- */
void leftISR() {
  // Direction logic (may need inversion – see test below)
  if (digitalRead(ENC_L_B) == LOW)
    leftCount--;
  else
    leftCount++;
}

void rightISR() {
  if (digitalRead(ENC_R_B) == LOW)
    rightCount++;
  else
```

```
  rightCount--;
}
```

A.2 PID tuning

```
#include <Wire.h>

/* -------- MOTOR PINS -------- */
#define M1A 6
#define M1B 9
#define M2A 10
#define M2B 11

/* -------- ENCODERS -------- */
#define ENCODER1_A 2
#define ENCODER1_B 4
#define ENCODER2_A 3
#define ENCODER2_B 5

volatile long count1 = 0;
volatile long count2 = 0;

void encoder1ISR() {
 if (digitalRead(ENCODER1_B)) count1--;
 else count1++;
}
void encoder2ISR() {
 if (digitalRead(ENCODER2_B)) count2++;
 else count2--;
}

/* -------- MPU -------- */
#define MPU_ADDR 0x68

int16_t ax, ay, az, gx, gy, gz;
```

```
float pitch = 0;
unsigned long lastTime = 0;

#define ALPHA 0.98

void initMPU() {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x6B);
  Wire.write(0x00);
  Wire.endTransmission();
}

void readMPU() {
  Wire.beginTransmission(MPU_ADDR);
  Wire.write(0x3B);
  Wire.endTransmission(false);
  Wire.requestFrom(MPU_ADDR, 14, true);

  ax = Wire.read()<<8 | Wire.read();
  ay = Wire.read()<<8 | Wire.read();
  az = Wire.read()<<8 | Wire.read();
  Wire.read(); Wire.read();
  gx = Wire.read()<<8 | Wire.read();
  gy = Wire.read()<<8 | Wire.read();
  gz = Wire.read()<<8 | Wire.read();
}

float getPitchAccel() {
  return atan2((float)ay, (float)az) * 180.0 / PI;
}

float getGyroRate() {
  return (float)gx / 131.0;
```

```
}

void updatePitch() {
  unsigned long now = millis();
  float dt = (now - lastTime) / 1000.0;
  lastTime = now;

  readMPU();

  float accel = getPitchAccel();
  float gyro  = getGyroRate();

  pitch = ALPHA * (pitch + gyro * dt) + (1 - ALPHA) * accel;

  //  Smooth value for cleaner graph
  pitch = round(pitch * 10) / 10.0;
}

/* -------- PID -------- */
float Kp = 5;
float Ki = 7;
float Kd = 2;//2.1

float setpoint = -4;
float error, prevError = 0;
float integral = 0.80;
float output;

/* -------- MOTOR SPEED TRACK -------- */
int leftSpeed = 0;
int rightSpeed = 0;

/* -------- MOTOR CONTROL -------- */
void setMotor(int speed) {
```

```
  speed = constrain(speed, -255, 255);

  leftSpeed = speed;
  rightSpeed = speed;

  if (speed > 0) {
    analogWrite(M1A, speed);
    analogWrite(M1B, 0);

    analogWrite(M2A, 0);
    analogWrite(M2B, speed);
  }
  else {
    int s = -speed;

    analogWrite(M1A, 0);
    analogWrite(M1B, s);

    analogWrite(M2A, s);
    analogWrite(M2B, 0);
  }
}

/* -------- SETUP -------- */
void setup() {
  Serial.begin(9600);

  pinMode(M1A, OUTPUT);
  pinMode(M1B, OUTPUT);
  pinMode(M2A, OUTPUT);
  pinMode(M2B, OUTPUT);

  pinMode(ENCODER1_A, INPUT_PULLUP);
```

```
  pinMode(ENCODER1_B, INPUT_PULLUP);
  pinMode(ENCODER2_A, INPUT_PULLUP);
  pinMode(ENCODER2_B, INPUT_PULLUP);

  attachInterrupt(digitalPinToInterrupt(ENCODER1_A), encoder1ISR, RISING);
  attachInterrupt(digitalPinToInterrupt(ENCODER2_A), encoder2ISR, RISING);

  Wire.begin();
  initMPU();
  delay(100);

  readMPU();
  pitch = getPitchAccel();
  lastTime = millis();

  Serial.println("Pitch,Setpoint,Upper,Lower,Output"); // header for clarity
}

/* -------- LOOP -------- */
void loop() {

  updatePitch();

  // -------- DEAD ZONE ±2 DEG --------
  if (pitch > -4 && pitch < -3) {

    error = 0;
    integral = 0;
    output = 0;

    setMotor(0);
  }
  else {
```

```
  error = pitch - setpoint;
  integral += error;

  // Anti-windup
  integral = constrain(integral, -100, 100);

  float derivative = error - prevError;

  output = Kp * error + Ki * integral + Kd * derivative;

  prevError = error;

  setMotor(-output);
 }

 // -------- GRAPH OUTPUT --------
 Serial.print(pitch);
 Serial.print(",");
 Serial.print(setpoint);
 Serial.print(",");
 Serial.print(2);    // upper limit
 Serial.print(",");
 Serial.print(-2);   // lower limit
 Serial.print(",");
 Serial.println(output);

 delay(5);
}
```

REFERENCES

[1] Nasir, A. N. K., Ahmad, M. A., and Ismail, R. (2010). “Control of a two-wheeled self-balancing robot using PID controller.” *International Conference on Mechatronics and Automation*.

[2] Pathak, K., Franch, J., and Agrawal, S. K. (2005). “Velocity and position control of a wheeled inverted pendulum by partial feedback linearization.” *IEEE Transactions on Robotics*.

[3] Arduino (2023). “Arduino Uno technical specifications and programming guide.” *Arduino Documentation*.

[4] Åström, K. J., and Hägglund, T. (1995). “PID controllers: Theory, design, and tuning.” *Instrument Society of America*.

[5] Kumar, A., and Verma, P. (2019). “Analysis of PID control for self-balancing robot.” *International Journal of Control Systems*.